Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 3: Python Strings

Practice Questions with Python Code Answers

Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. Break, Continue, Pass
2. Creating Strings & Indexing
3. String Slicing
4. Edit and Delete a String
5. Operations on Strings
6. Case Conversion Functions
7. count(), find(), index()
8. startswith() and endswith()
9. format() and f-strings
10. isalnum(), isalpha(), isdigit(), isidentifier()
11. split() and join()
12. replace() Function
13. strip() Function

78 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad

+91 77220 82226 Chh. Sambhajinagar

+91 77220 82227 Nashik

+91 77220 82224 Hadapsar, Pune

Session 3: Python Strings - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: Break, Continue, Pass, Creating Strings & Indexing, String Slicing, Edit and Delete a String, Operations on Strings, Case Conversion Functions, count(), find(), index(), startswith() and endswith(), format() and f-strings, isalnum(), isalpha(), isdigit(), isidentifier(), split() and join(), replace() Function, strip() Function
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. Break, Continue, Pass

Q1. Stop at Out-of-Stock Item

BEGINNER

A billing machine scans products ["Soap", "Oil", "OUT", "Sugar", "Tea"]. When it reads "OUT" it must stop scanning. Use break.

Solution:

```
products = ["Soap", "Oil", "OUT", "Sugar", "Tea"]

for item in products:
    if item == "OUT":
        break
    print(item)
```

Output:

```
Soap
Oil
```

Q2. Skip Absent Students

BEGINNER

Roll call status is ["P", "A", "P", "P", "A"] for roll numbers 1 to 5. Print only the roll numbers of present students using continue.

Solution:

```
status = ["P", "A", "P", "P", "A"]

for i in range(len(status)):
    if status[i] == "A":
        continue
    print(i + 1)
```

Output:

```
1
3
4
```

Q3. Feature Not Ready Yet (pass)

INTERMEDIATE

A food app has payment modes ["UPI", "Card", "Wallet"]. The Wallet feature is not built yet. Use pass as a placeholder for Wallet and print a message for the other modes.

Solution:

```
modes = ["UPI", "Card", "Wallet"]

for mode in modes:
    if mode == "Wallet":
        pass        # TODO: build wallet payment later
    else:
        print(mode, "payment ready")
```

Output:

```
UPI payment ready
Card payment ready
```

Q4. Search Roll Number (for-else)

INTERMEDIATE

Search for roll number 118 in the list [101, 105, 112, 118, 120]. If found print "Found at position" with its position and stop. If not found, print "Not found" using the else part of the for loop.

Solution:

```
rolls = [101, 105, 112, 118, 120]
search = 118

for i in range(len(rolls)):
    if rolls[i] == search:
        print("Found at position", i)
        break
else:
    print("Not found")
```

Output:

```
Found at position 3
```

Q5. Process Bank Transactions

ADVANCED

Process transactions [500, -200, 1500, 0, 700, "END", 900]. Skip negative and zero values (invalid) using continue, stop when "END" is reached using break, and print the total of valid transactions.

Solution:

```
transactions = [500, -200, 1500, 0, 700, "END", 900]
total = 0

for t in transactions:
    if t == "END":
        break
    if t <= 0:
        continue
    total += t

print(total)
```

Output:

```
2700
```

Q6. First 5 Multiples of 7

ADVANCED

Using a while loop over numbers 1 to 100, collect only the first 5 numbers divisible by 7. Use continue to skip non-multiples and break once 5 are found.

Solution:

```
n = 0
found = 0

while n < 100:
    n += 1
    if n % 7 != 0:
        continue
    print(n)
    found += 1
    if found == 5:
        break
```

Output:

```
7
14
21
28
35
```

2. Creating Strings & Indexing

Q7. Create Address Strings

BEGINNER

Store a company name with single quotes, a tagline with double quotes, and a 3-line office address with triple quotes. Print all three.

Solution:

```
company = 'Bikkad IT'
tagline = "Dream Big! Start Small!! Act Now!!!"
address = '''Office No. 5,
Pimpri-Chinchwad,
Pune - 411018'''

print(company)
print(tagline)
print(address)
```

Output:

```
Bikkad IT
Dream Big! Start Small!! Act Now!!!
Office No. 5,
Pimpri-Chinchwad,
Pune - 411018
```

Q8. First and Last Letter

BEGINNER

For the brand name "Flipkart", print the first character using index 0 and the last character using index -1.

Solution:

```
brand = "Flipkart"

print(brand[0])
print(brand[-1])
```

Output:

```
F
t
```

Q9. Employee Initials

INTERMEDIATE

First name is "Swapnil" and last name is "Jadhav". Use indexing to create the initials "S.J.".

Solution:

```
first = "Swapnil"
last = "Jadhav"

initials = first[0] + "." + last[0] + "."
print(initials)
```

Output:

```
S.J.
```

Q10. Check Digit of a Vehicle Number

INTERMEDIATE

For the vehicle number "MH12AB4567" print the 1st character, the 5th character and the last character using positive and negative indexing. Also print the length.

Solution:

```
vehicle = "MH12AB4567"

print(vehicle[0])
print(vehicle[4])
print(vehicle[-1])
print(len(vehicle))
```

Output:

```
M
A
7
10
```

Q11. Index Map of a Word

ADVANCED

For the word "PYTHON", print each character with its positive index and its matching negative index using a loop.

Solution:

```
word = "PYTHON"
n = len(word)

for i in range(n):
    print(word[i], i, i - n)
```

Output:

```
P 0 -6
Y 1 -5
T 2 -4
H 3 -3
O 4 -2
N 5 -1
```

Q12. Palindrome Using Indexing

ADVANCED

Check whether the word "malayalam" is a palindrome by comparing word[i] with word[-(i+1)] in a loop (without slicing).

Solution:

```
word = "malayalam"
is_palindrome = True

for i in range(len(word) // 2):
    if word[i] != word[-(i + 1)]:
        is_palindrome = False
        break

print(is_palindrome)
```

Output:

```
True
```

3. String Slicing

Q13. Year from a Date

BEGINNER

A date is stored as "17-10-2026". Use slicing to extract the day, month and year.

Solution:

```
date = "17-10-2026"

day = date[0:2]
month = date[3:5]
year = date[6:]

print(day)
print(month)
print(year)
```

Output:

```
17
10
2026
```

Q14. Vehicle Registration Parts

BEGINNER

From "MH12AB4567" slice the state code (MH), the RTO code (12), the series (AB) and the number (4567).

Solution:

```
vehicle = "MH12AB4567"

state = vehicle[:2]
rto = vehicle[2:4]
series = vehicle[4:6]
number = vehicle[6:]

print(state)
print(rto)
print(series)
print(number)
```

Output:

```
MH
12
AB
4567
```

Q15. Mask a Card Number

INTERMEDIATE

A sample card number is "4321876512349876". Show only the last 4 digits and replace the rest with X, like "XXXXXXXXXXXX9876".

Solution:

```
card = "4321876512349876"

masked = "X" * (len(card) - 4) + card[-4:]
print(masked)
```

Output:

```
XXXXXXXXXXXX9876
```

Q16. Reverse and Palindrome

INTERMEDIATE

Reverse the word "level" and "python" using slicing with step -1 and check whether each is a palindrome.

Solution:

```
word1 = "level"
word2 = "python"

print(word1[::-1])
print(word1 == word1[::-1])

print(word2[::-1])
print(word2 == word2[::-1])
```

Output:

```
level
True
nohtyp
False
```

Q17. Username and Domain from Email

ADVANCED

From the email "meera.kulkarni@bikkadit.com" use find() and slicing to extract the username (before @) and the domain name (between @ and the last dot).

Solution:

```
email = "meera.kulkarni@bikkadit.com"

at_pos = email.find("@")
dot_pos = email.rfind(".")

username = email[:at_pos]
domain = email[at_pos + 1:dot_pos]

print(username)
print(domain)
```

Output:

```
meera.kulkarni
bikkadit
```

Q18. Alternate Letters and Rotation

ADVANCED

For "BIKKADIT": print every alternate character, print the characters at odd positions, and rotate the string left by 3 positions ("KADITBIK") using slicing.

Solution:

```
text = "BIKKADIT"

print(text[::2])
print(text[1::2])

k = 3
rotated = text[k:] + text[:k]
print(rotated)
```

Output:

```
BKAI
IKDT
KADITBIK
```

4. Edit and Delete a String

Q19. Fix the City Name

BEGINNER

A city is typed as "pune". Strings cannot be changed directly, so create a new string "Pune" by joining an uppercase "P" with the rest of the word.

Solution:

```
city = "pune"

city = "P" + city[1:]
print(city)
```

Output:

```
Pune
```

Q20. Proof that Strings are Immutable

BEGINNER

Store "Hello" in a variable, print its id(), then add " World" to it and print the id() again. The id changes because a NEW string is created.

Solution:

```
msg = "Hello"
old_id = id(msg)

msg = msg + " World"
new_id = id(msg)

print(msg)
print(old_id == new_id)
```

Output:

```
Hello World
False
```

Q21. Replace a Character by Position

INTERMEDIATE

In the product code "AB7-2X9" the character at index 3 ("-") should become "/". Create the new code using slicing.

Solution:

```
code = "AB7-2X9"

index = 3
new_code = code[:index] + "/" + code[index + 1:]
print(new_code)
```

Output:

```
AB7/2X9
```

Q22. Format a Mobile Number

INTERMEDIATE

Insert a hyphen in the middle of the mobile number "9876543210" to get "98765-43210".

Solution:

```
mobile = "9876543210"

formatted = mobile[:5] + "-" + mobile[5:]
print(formatted)
```

Output:

```
98765-43210
```

Q23. Remove a Character at Any Index

ADVANCED

Remove the character at index 4 from "Bikkkad" (an extra k typed by mistake) to get "Bikkad". Then remove all digits from "Ra9hu7l" by building a new string in a loop.

Solution:

```
word = "Bikkkad"
index = 4

fixed = word[:index] + word[index + 1:]
print(fixed)

name = "Ra9hu7l"
clean = ""
for ch in name:
    if not ch.isdigit():
        clean = clean + ch
print(clean)
```

Output:

```
Bikkad
Rahul
```

Q24. Delete a String Variable

ADVANCED

Create a temporary OTP string "482913", use it, then delete the variable with del. Check whether the name still exists using "otp" in dir().

Solution:

```
otp = "482913"
print(otp)
print("otp" in dir())

del otp
print("otp" in dir())
# print(otp)  would now give NameError
```

Output:

```
482913
True
False
```

5. Operations on Strings

Q25. Full Name

BEGINNER

Join first name "Tanvi" and last name "More" with a space using + and print the full name.

Solution:

```
first = "Tanvi"
last = "More"

full_name = first + " " + last
print(full_name)
```

Output:

```
Tanvi More
```

Q26. Report Heading Line

BEGINNER

Print a heading "MONTHLY SALES REPORT" with a line of 25 "=" characters above and below it using the * operator.

Solution:

```
line = "=" * 25

print(line)
print("MONTHLY SALES REPORT")
print(line)
```

Output:

```
=========================
MONTHLY SALES REPORT
=========================
```

Q27. Email Has @ and .

INTERMEDIATE

Check whether the email "amit.p@yahoo.in" contains both "@" and "." using the in operator.

Solution:

```
email = "amit.p@yahoo.in"

valid = "@" in email and "." in email
print(valid)
```

Output:

```
True
```

Q28. Password Length Rule

INTERMEDIATE

A website requires a password of at least 8 characters. Check the passwords "pune123" and "Nashik@2026" using len().

Solution:

```
pass1 = "pune123"
pass2 = "Nashik@2026"

print(len(pass1))
print(len(pass1) >= 8)
print(len(pass2))
print(len(pass2) >= 8)
```

Output:

```
7
False
11
True
```

Q29. Dotted Receipt Lines

ADVANCED

Print receipt lines where the item name is followed by dots so that every line is 25 characters long before the price. Items: Paneer Tikka 260, Naan 40, Lassi 70. Use len() and *.

Solution:

```
width = 25

item = "Paneer Tikka"
print(item + "." * (width - len(item)) + "260")

item = "Naan"
print(item + "." * (width - len(item)) + "40")

item = "Lassi"
print(item + "." * (width - len(item)) + "70")
```

Output:

```
Paneer Tikka.............260
Naan.....................40
Lassi....................70
```

Q30. Alphabetical Comparison of Names

ADVANCED

Compare the names "Sameer", "Anjali" and "Kiran" using <, min() and max() to find which comes first and last alphabetically. Then compare "apple" and "Apple".

Solution:

```
a = "Sameer"
b = "Anjali"
c = "Kiran"

print(b < a)
print(min(a, b, c))
print(max(a, b, c))

print("apple" == "Apple")
print("Apple" < "apple")
```

Output:

```
True
Anjali
Sameer
False
True
```

6. Case Conversion Functions

Q31. Product Code in Uppercase

BEGINNER

A product code is typed as "lap-hp-15s". Convert it to uppercase.

Solution:

```
code = "lap-hp-15s"

print(code.upper())
```

Output:

```
LAP-HP-15S
```

Q32. Customer Name in Title Case

BEGINNER

A customer name is typed as "rohit vijay pawar". Convert it to title case.

Solution:

```
name = "rohit vijay pawar"

print(name.title())
```

Output:

```
Rohit Vijay Pawar
```

Q33. Case-Insensitive Login

INTERMEDIATE

The stored username is "BikkadAdmin" and the user typed "bikkadadmin". Compare them ignoring case using lower().

Solution:

```
stored = "BikkadAdmin"
typed = "bikkadadmin"

print(stored == typed)
print(stored.lower() == typed.lower())
```

Output:

```
False
True
```

Q34. Feedback Sentence

INTERMEDIATE

Customer feedback is "tHE fOOD was GREAT". Use capitalize() to make only the first letter capital, and swapcase() to see the reversed case.

Solution:

```
feedback = "tHE fOOD was GREAT"

print(feedback.capitalize())
print(feedback.swapcase())
```

Output:

```
The food was great
The Food WAS great
```

Q35. Count Uppercase and Lowercase

ADVANCED

Count the uppercase and lowercase letters in "Bikkad IT Institute Pune" using a loop with isupper() and islower().

Solution:

```
text = "Bikkad IT Institute Pune"
upper_count = 0
lower_count = 0

for ch in text:
    if ch.isupper():
        upper_count += 1
    elif ch.islower():
        lower_count += 1

print(upper_count)
print(lower_count)
```

Output:

```
5
16
```

Q36. Clean Messy City Entries

ADVANCED

A survey has city entries ["pune", "PUNE", " Pune ", "nashik", "NASHIK"]. Clean each entry with strip() and title(), store the clean values in a set, and print the unique cities.

Solution:

```
entries = ["pune", "PUNE", " Pune ", "nashik", "NASHIK"]
unique_cities = set()

for e in entries:
    clean = e.strip().title()
    unique_cities.add(clean)

print(sorted(unique_cities))
```

Output:

```
['Nashik', 'Pune']
```

7. count(), find(), index()

Q37. Count a Letter

BEGINNER

Count how many times the letter "s" appears in "Mississippi".

Solution:

```
word = "Mississippi"

print(word.count("s"))
```

Output:

```
4
```

Q38. Position of @ in Email

BEGINNER

Find the position of "@" in the email "sales@bikkadit.com" using find().

Solution:

```
email = "sales@bikkadit.com"

print(email.find("@"))
```

Output:

```
5
```

Q39. Word Count in a Review

INTERMEDIATE

A review says "good food, good service, good price". Count how many times the word "good" appears and find its first position.

Solution:

```
review = "good food, good service, good price"

print(review.count("good"))
print(review.find("good"))
```

Output:

```
3
0
```

Q40. Safe Search with find()

INTERMEDIATE

Search for "refund" in the message "Your order has been delivered". Since find() returns -1 when the word is missing, use if-else to print a friendly message.

Solution:

```
message = "Your order has been delivered"
pos = message.find("refund")

if pos == -1:
    print("No refund information")
else:
    print("Refund found at", pos)
```

Output:

```
No refund information
```

Q41. All Positions of a Word

ADVANCED

Find ALL positions where "na" appears in "banana bandana" using find() with a start position inside a while loop.

Solution:

```
text = "banana bandana"
target = "na"
start = 0

while True:
    pos = text.find(target, start)
    if pos == -1:
        break
    print(pos)
    start = pos + 1
```

Output:

```
2
4
12
```

Q42. Extract Text Between Brackets

ADVANCED

From the log line "Payment received [Order#4521] at 10:45" extract the text inside the square brackets using index().

Solution:

```
log = "Payment received [Order#4521] at 10:45"

start = log.index("[")
end = log.index("]")

order = log[start + 1:end]
print(order)
```

Output:

```
Order#4521
```

8. startswith() and endswith()

Q43. Is it a PDF?

BEGINNER

Check whether the file "Python_Notes.pdf" ends with ".pdf".

Solution:

```
file_name = "Python_Notes.pdf"

print(file_name.endswith(".pdf"))
```

Output:

```
True
```

Q44. Indian Mobile Number

BEGINNER

Check whether the phone number "+919823456710" starts with "+91".

Solution:

```
phone = "+919823456710"

print(phone.startswith("+91"))
```

Output:

```
True
```

Q45. Filter Image Files

INTERMEDIATE

From the files ["logo.png", "notes.docx", "photo.jpg", "data.csv", "banner.jpeg"] print only image files. Pass a tuple (".png", ".jpg", ".jpeg") to endswith().

Solution:

```
files = ["logo.png", "notes.docx", "photo.jpg",
         "data.csv", "banner.jpeg"]

for f in files:
    if f.endswith((".png", ".jpg", ".jpeg")):
        print(f)
```

Output:

```
logo.png
photo.jpg
banner.jpeg
```

Q46. Secure Website Check

INTERMEDIATE

Check whether the URLs "https://bikkadit.com" and "http://example.org" are secure (start with "https").

Solution:

```
url1 = "https://bikkadit.com"
url2 = "http://example.org"

print(url1.startswith("https"))
print(url2.startswith("https"))
```

Output:

```
True
False
```

Q47. Classify Transaction IDs

ADVANCED

Transaction IDs are ["UPI8841", "NEFT2201", "IMPS5532", "UPI1190", "CASH001"]. Use startswith() with if-elif to print the payment type of each.

Solution:

```
ids = ["UPI8841", "NEFT2201", "IMPS5532", "UPI1190", "CASH001"]

for t in ids:
    if t.startswith("UPI"):
        print(t, "UPI Payment")
    elif t.startswith("NEFT"):
        print(t, "NEFT Transfer")
    elif t.startswith("IMPS"):
        print(t, "IMPS Transfer")
    else:
        print(t, "Other")
```

Output:

```
UPI8841 UPI Payment
NEFT2201 NEFT Transfer
IMPS5532 IMPS Transfer
UPI1190 UPI Payment
CASH001 Other
```

Q48. Pune Vehicles Only

ADVANCED

From ["MH12AB1234", "MH14CD5678", "MH12EF9012", "KA01GH3456"] print vehicles from Pune (start with "MH12"). Also show startswith() with a start index: check if characters from index 2 start with "12".

Solution:

```
vehicles = ["MH12AB1234", "MH14CD5678",
            "MH12EF9012", "KA01GH3456"]

for v in vehicles:
    if v.startswith("MH12"):
        print(v)

print("MH12AB1234".startswith("12", 2))
```

Output:

```
MH12AB1234
MH12EF9012
True
```

9. format() and f-strings

Q49. Course Welcome Message

BEGINNER

Use format() to print "Welcome Snehal to the Python course" with the name and course stored in variables.

Solution:

```
name = "Snehal"
course = "Python"

message = "Welcome {} to the {} course".format(name, course)
print(message)
```

Output:

```
Welcome Snehal to the Python course
```

Q50. Bill with f-string

BEGINNER

Quantity is 3 and price is 120. Use an f-string to print "Total amount: 360" by calculating inside the f-string.

Solution:

```
quantity = 3
price = 120

print(f"Total amount: {quantity * price}")
```

Output:

```
Total amount: 360
```

Q51. Price with 2 Decimals

INTERMEDIATE

A product costs 1499 and has a 7.5% discount. Print the discounted price, and then the same price with exactly 2 decimal places using {:.2f} in an f-string and in format().

Solution:

```
price = 1499
final = price * 0.925

print(final)
print(f"{final:.2f}")
print("{:.2f}".format(final))
```

Output:

```
1386.575
1386.58
1386.58
```

Q52. Offer Letter Line

INTERMEDIATE

Use format() with positional ({0}, {1}) and named ({role}) placeholders to print: "Dear Arjun, you are selected as Data Analyst. Welcome Arjun!"

Solution:

```
text = "Dear {0}, you are selected as {role}. Welcome {0}!"
letter = text.format("Arjun", role="Data Analyst")
print(letter)
```

Output:

```
Dear Arjun, you are selected as Data Analyst. Welcome Arjun!
```

Q53. Aligned Marks Table

ADVANCED

Print a marks table for 3 students with names left-aligned in 10 spaces and marks right-aligned in 6 spaces using {:<10} and {:>6}.

Solution:

```
print(f"{'Name':<10}{'Marks':>6}")
print(f"{'Aditi':<10}{91:>6}")
print(f"{'Mohan':<10}{78:>6}")
print(f"{'Zaid':<10}{85:>6}")
```

Output:

```
Name       Marks
Aditi         91
Mohan         78
Zaid          85
```

Q54. Professional Number Formatting

ADVANCED

Format: sales 1250000 with thousand separators {:,}, a growth rate 0.1275 as a percentage with 1 decimal {:.1%}, and invoice number 42 with zero padding to 5 digits {:05d}.

Solution:

```
sales = 1250000
growth = 0.1275
invoice = 42

print(f"{sales:,}")
print(f"{growth:.1%}")
print(f"INV-{invoice:05d}")
```

Output:

```
1,250,000
12.8%
INV-00042
```

10. isalnum(), isalpha(), isdigit(), isidentifier()

Q55. Valid PIN Code

BEGINNER

Check whether "411018" and "41101A" contain only digits using isdigit().

Solution:

```
pin1 = "411018"
pin2 = "41101A"

print(pin1.isdigit())
print(pin2.isdigit())
```

Output:

```
True
False
```

Q56. Name Contains Only Letters

BEGINNER

Check "Prakash" and "Prakash1" using isalpha().

Solution:

```
name1 = "Prakash"
name2 = "Prakash1"

print(name1.isalpha())
print(name2.isalpha())
```

Output:

```
True
False
```

Q57. Username Rules

INTERMEDIATE

A username can contain only letters and digits. Check "vikas2026", "vikas_2026" and "vikas 2026" using isalnum().

Solution:

```
print("vikas2026".isalnum())
print("vikas_2026".isalnum())
print("vikas 2026".isalnum())
```

Output:

```
True
False
False
```

Q58. Valid Variable Names

INTERMEDIATE

A student wrote these variable names: "total_marks", "2nd_rank", "_temp", "student-name". Use isidentifier() in a loop to check each one.

Solution:

```
names = ["total_marks", "2nd_rank", "_temp", "student-name"]

for n in names:
    print(n, n.isidentifier())
```

Output:

```
total_marks True
2nd_rank False
_temp True
student-name False
```

Q59. Password Strength Checker

ADVANCED

A strong password must have at least 8 characters, one uppercase letter, one lowercase letter, one digit and one special character. Check "Bikkad@2026" using a loop and is-methods.

Solution:

```
password = "Bikkad@2026"
has_upper = False
has_lower = False
has_digit = False
has_special = False

for ch in password:
    if ch.isupper():
        has_upper = True
    elif ch.islower():
        has_lower = True
    elif ch.isdigit():
        has_digit = True
    elif not ch.isalnum():
        has_special = True

strong = (len(password) >= 8 and has_upper and has_lower
          and has_digit and has_special)
print(strong)
```

Output:

```
True
```

Q60. Indian Mobile Number Validation

ADVANCED

A valid Indian mobile number has exactly 10 digits and starts with 6, 7, 8 or 9. Validate "9823456710", "5823456710" and "98234A6710".

Solution:

```
numbers = ["9823456710", "5823456710", "98234A6710"]

for num in numbers:
    valid = len(num) == 10 and num.isdigit() and num[0] in "6789"
    print(num, valid)
```

Output:

```
9823456710 True
5823456710 False
98234A6710 False
```

11. split() and join()

Q61. Count Words

BEGINNER

Split the sentence "Python is easy to learn" into words and print the list and the number of words.

Solution:

```
sentence = "Python is easy to learn"

words = sentence.split()
print(words)
print(len(words))
```

Output:

```
['Python', 'is', 'easy', 'to', 'learn']
5
```

Q62. Blog URL Slug

BEGINNER

Convert the list ["learn", "python", "in", "marathi"] into a URL slug "learn-python-in-marathi" using join().

Solution:

```
words = ["learn", "python", "in", "marathi"]

slug = "-".join(words)
print(slug)
```

Output:

```
learn-python-in-marathi
```

Q63. Read a CSV Record

INTERMEDIATE

A CSV line is "Amol,24,Pune,Data Analyst". Split it by comma and print each field separately.

Solution:

```
record = "Amol,24,Pune,Data Analyst"

fields = record.split(",")
print(fields)
print(fields[0])
print(fields[1])
print(fields[2])
print(fields[3])
```

Output:

```
['Amol', '24', 'Pune', 'Data Analyst']
Amol
24
Pune
Data Analyst
```

Q64. Reverse the Word Order

INTERMEDIATE

Reverse the order of words in "I love coding in Python" to get "Python in coding love I".

Solution:

```
sentence = "I love coding in Python"

words = sentence.split()
reversed_words = words[::-1]
result = " ".join(reversed_words)
print(result)
```

Output:

```
Python in coding love I
```

Q65. Initials from Full Name

ADVANCED

From the full name "Neha Suresh Kulkarni" create the initials "N.S.K." using split(), a loop and join().

Solution:

```
full_name = "Neha Suresh Kulkarni"

parts = full_name.split()
letters = []
for p in parts:
    letters.append(p[0])

initials = ".".join(letters) + "."
print(initials)
```

Output:

```
N.S.K.
```

Q66. Parse a Settings String

ADVANCED

An app stores settings as "theme=dark;lang=marathi;font=14". Split it into pairs and then into keys and values to build a dictionary.

Solution:

```
settings_text = "theme=dark;lang=marathi;font=14"
settings = {}

for pair in settings_text.split(";"):
    key, value = pair.split("=")
    settings[key] = value

print(settings)
print(settings["lang"])
```

Output:

```
{'theme': 'dark', 'lang': 'marathi', 'font': '14'}
marathi
```

12. replace() Function

Q67. Change the City

BEGINNER

A customer moved. Replace "Mumbai" with "Pune" in the address "Flat 12, Andheri, Mumbai".

Solution:

```
address = "Flat 12, Andheri, Mumbai"

new_address = address.replace("Mumbai", "Pune")
print(new_address)
```

Output:

```
Flat 12, Andheri, Pune
```

Q68. Remove Spaces from Phone

BEGINNER

Remove all spaces from the phone number "98 23 45 67 10".

Solution:

```
phone = "98 23 45 67 10"

clean = phone.replace(" ", "")
print(clean)
```

Output:

```
9823456710
```

Q69. Censor a Bad Word

INTERMEDIATE

Replace the word "stupid" with "******" in the comment "This is a stupid idea, really stupid".

Solution:

```
comment = "This is a stupid idea, really stupid"

clean = comment.replace("stupid", "*" * len("stupid"))
print(clean)
```

Output:

```
This is a ****** idea, really ******
```

Q70. Replace Only the First Occurrence

INTERMEDIATE

In "sale sale sale" replace only the first "sale" with "SUPER" using the count argument.

Solution:

```
text = "sale sale sale"

print(text.replace("sale", "SUPER", 1))
```

Output:

```
SUPER sale sale
```

Q71. Clean a Currency Value

ADVANCED

A price is stored as "Rs. 1,25,000.50". Remove "Rs.", commas and spaces using chained replace() calls, convert it to float and add 1000.

Solution:

```
price_text = "Rs. 1,25,000.50"

clean = price_text.replace("Rs.", "")
clean = clean.replace(",", "")
clean = clean.replace(" ", "")
print(clean)

price = float(clean)
print(price + 1000)
```

Output:

```
125000.50
126000.5
```

Q72. SMS Template for Many Customers

ADVANCED

An SMS template is "Hi <NAME>, your bill of Rs.<AMT> is due". Using a loop and replace(), create messages for the customers ["Ravi", "Sana"] with amounts [540, 1200].

Solution:

```
template = "Hi <NAME>, your bill of Rs.<AMT> is due"
names = ["Ravi", "Sana"]
amounts = [540, 1200]

for i in range(len(names)):
    msg = template.replace("<NAME>", names[i])
    msg = msg.replace("<AMT>", str(amounts[i]))
    print(msg)
```

Output:

```
Hi Ravi, your bill of Rs.540 is due
Hi Sana, your bill of Rs.1200 is due
```

13. strip() Function

Q73. Clean User Name

BEGINNER

A user typed " Manasi " with extra spaces. Remove spaces from both sides and print it inside brackets to see the difference.

Solution:

```
name = "   Manasi   "

print("[" + name + "]")
print("[" + name.strip() + "]")
```

Output:

```
[   Manasi   ]
[Manasi]
```

Q74. Remove Leading Zeros

BEGINNER

An employee ID is "000452". Remove the leading zeros using lstrip("0").

Solution:

```
emp_id = "000452"

print(emp_id.lstrip("0"))
```

Output:

```
452
```

Q75. Clean the End of a Line

INTERMEDIATE

A line read from a file is "Total sales: 45000.\n". Remove the newline and the full stop from the right side using rstrip().

Solution:

```
line = "Total sales: 45000.\n"

clean = line.rstrip("\n.")
print(clean)
```

Output:

```
Total sales: 45000
```

Q76. Remove Decoration Characters

INTERMEDIATE

A banner text is "***DIWALI SALE***". Remove the stars from both sides using strip("*").

Solution:

```
banner = "***DIWALI SALE***"

print(banner.strip("*"))
```

Output:

```
DIWALI SALE
```

Q77. Clean Form Entries

ADVANCED

Form names are [" aarti ", "MAHESH ", " kunal"]. Clean each name with strip() and title() and store them in a new list.

Solution:

```
raw_names = ["  aarti ", "MAHESH  ", "  kunal"]
clean_names = []

for n in raw_names:
    clean_names.append(n.strip().title())

print(clean_names)
```

Output:

```
['Aarti', 'Mahesh', 'Kunal']
```

Q78. strip() Removes Characters, Not Words

ADVANCED

Try "https://shop.com".strip("https:/") and see that it removes CHARACTERS from both ends (it also eats the letters "sh" of "shop"). Use removeprefix() (Python 3.9+) to remove the exact prefix correctly.

Solution:

```
url = "https://shop.com"

print(url.strip("https:/"))
print(url.removeprefix("https://"))
```

Output:

```
op.com
shop.com
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026